

PJ2 实验文档

一、 代码基本结构

1、mnist 手写数字识别

整个网络我采用了两层的卷积层和两层的线性层。对于 mnist 手写数字识别，即使不使用卷积其正确率也已经很高，因此在这个任务中我并没有叠加很多的卷积层，最后正确率也达到了 99%。神经网络的具体结构是：卷积 1→激活函数→最大池化 1→卷积 2→激活函数→最大值池化 2→展平→全连接层 1→激活函数→全连接层 2。由于 pytorch 的 `nn.functional.cross_entropy` 不需要显式在最后一层使用 `softmax`，因此我直接输出了 logits 作为结果，在训练中进一步计算 loss。

在这个任务中我并没有使用复杂的数据增强技巧，仅仅使用了 `transforms.Normalize((0.1307, ), (0.3081, ))` 将数据进行归一化

(normalization)，使得原始数据以 0 为均值，以 1 为标准差。这种方式使得数据分布更为集中，减少异常值对于模型训练的干扰，最终可以使得模型的学习更加平稳。与此同时，在这个任务中我选择了 `tanh` 作为激活函数，但是 `tanh` 函数在输入值过大或过小时梯度极小，而在输入值趋近于 0 时梯度较大。因此当输入值在 0 左右时模型的训练会比较高效。这种归一化能够使得数据更加符合 `tanh` 函数的特性，因此我加上了它。

2、cifar-10 图像分类

我的第一版结构基本保留了 mnist 的结构，即采用了卷积层和全连接层，但是根据 cifar-10 的图片大小和通道数对于具体结构进行了修改。由于 cifar-10 的原图大小为 32*32*32，通道数是 mnist 的三倍，大小也大了一些，因此我增加了一层卷积层，希望卷积层能够更好地提取表征。同时我也增加了一层线性层，防止线性层的维度跨越太大导致分类效果降低。

同时我也进行了数据增强，使用了 `transforms.RandomCrop(32, padding=4)`。这个操作是先在原始图片周围填充 4 个像素宽的区域，再随机裁剪出 32*32 的区域。这种操作符合原图的性质，即关键物体并不一定处在图片的正中间，从而调整中心点的位置并进行裁剪，以减少无关物体的干扰，让模型更好地关注到物体的表征。但是这种方法也可能将物体切去一半，从而增大了学习难度，提升了鲁棒性。

使用 `transforms.RandomHorizontalFlip()`，让原图有一半概率进行旋转，使得模型的学习不局限于正常视角，提升鲁棒性。

`transforms.ToTensor()` 这一步将原有的 [0, 255] 范围内的像素值放缩到 [0, 1] 范围，将数值的变化范围缩小，减少学习的难度。

但是，由于 ReLU 函数的特性：大于 0 的部分梯度为 1，小于 0 的部分梯度为 0，因此当像素值全部为正数时不能完全发挥 ReLU 函数的作用，因而再采用 `transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))` 将图片的像素值从 `[0, 1]` 统一调整到 `[-1, 1]`，加入负数的存在，符合 ReLU 函数的正负切换特性。

二、进一步调整实验参数

1、minst 手写数字识别

由于数据中没有提供测试集，我从原始数据集中划分了 20% 作为验证集。于此同时，模型训练完成后，我将模型在官网上的测试集进行了测试，并比较了它们之间的结果，得到了有趣的现象。

我先尝试了最简单的单层卷积层，单层线性层，即“1+1”结构，在图像增强方面，我加入了 `transforms.Normalize((0.1307, ), (0.3081, ))`，最终模型在验证集上的正确率达到了 98.12%。而“2+1”结构下，验证集上的正确率达到 98.6%。在去掉 `transforms.Normalize((0.1307, ), (0.3081, ))` 后，“1+1”结构的准确率仍然能够达到 98%。原因可能是这个数据集本身比较简单，是否进行这种简单的数据增强影响不大。但是，加入这个变换会使得训练时间显著增加，因此可以考虑不加。

在接下来的 7 次实验中，我没有加入 `transforms.Normalize((0.1307, ), (0.3081, ))`。我进一步尝试了“2+1”的方式，即增加一层卷积层，在其他参数相同的情况下，验证集上的正确率达到了 98.55% 和 98.72%。进一步尝试“2+2”，正确率达到了 99.08% 和 98.95%。之后我又尝试了加上 `transforms.Normalize((0.1307, ), (0.3081, ))`，结果反而有些降低，分别为 98.81% 和 98.94%。这进一步验证了我的想法。

我又将这些模型在官网上的测试集上进行了测试，发现验证集上表现更好的模型在测试集上表现并不一定更好。具体数据可以看下面的图片：

实验编号	卷积层数	线性层数	训练轮数	transform	正确率	在官网数据集上正确率	去掉的加回来，没去掉的去掉
1	1	1	20		98.12	98.3	94.11
2	2	1	20		98.6	98.88	96.28
3	1	1	20		97.88	97.96	86.98
4	1	1	20	去掉 <code>transforms.Normalize((0.1307,), (0.3081,))</code>	98.22	98.36	91.48
5	1	1	20	同上	98.35	98.53	92.19
6	1	1	20		98.25	98.34	92.15
7	2	1	20		98.55	98.63	98.6
8	2	1	20		98.72	98.67	98.56
9	2	2	20		99.08	98.83	98.72
10	2	2	20		98.95	99.02	99.04
11	2	2	20	加上	98.81	98.98	98.44
12	2	2	20	训练速度变慢了	98.94	99.03	98.82

不过从整体上来看，“2+2”结构，即两层卷积层两层线性层的结构表现往往更好。

为了测试数据集的鲁棒性，我在测试时设计与训练时相反的情形，即训练时使用了 `transforms.Normalize((0.1307, ), (0.3081, ))` 的在测试时去掉，反

之亦然。我仍然使用官网数据集进行测试，结果发现，“1+1”结构的模型准确率下降最严重，而2层卷积层的结构在数据上都没有明显差距，甚至出现了正确率更高的情况。这种现象可能是因为两层的神经网络学习到了更好的表征，使得模型的鲁棒性更好了。因此最终我的模型采用了“2+2”结构。

2、cifar-10

在开始时我仅仅使用了三层卷积层，其具体参数如下：

三层卷积层：通道数变化：3→16, 16→32, 32→64，最大池化 maxpool==2*2, 卷积核大小 kernel size=3*3。

三层线性层：维度变化：1024→512→120→10。

训练轮数：Epoch=20，优化器 optimizer=Adam，参数默认，使用 dropout。

在这样的参数下，我最终跑到了 75.02% 的正确率。

接着我尝试了去掉 dropout 进行实验，在其他参数相同的情况下，不使用 dropout 的准确率达到 77.65%。可以看出，dropout 在浅层网络上效果并不好。具体的原因分析我放在了 pj3 中。

由于正确率不到 80%，可能是现有的神经网络的学习能力不够，以及 20 轮的训练轮数下模型可能并没有完全学到数据集的特征。考虑到 cifar-10 的图片相对复杂，三层卷积层可能不够，因此我增加了一层卷积层，并调整了对应的线性层维度，并增加训练轮数至 50 次。为了进一步验证 dropout 在四层卷积层的网络结构上效果如何，我仍然添加了 dropout 层，最终准确率为 78.98%，结果仍然不够理想。因此在后续的实验中，我都没有加 dropout 层。去掉 dropout 层再次在相同的参数下进行实验，准确率达到 82.04%。我进一步增大了训练轮数至 100 轮，发现正确率达到 82.67%，已经接近饱和。

通过查阅文献，我了解到在不使用如 resnet 等更深层的神经网络，在浅层网络上 cifar-10 的正确率的确难以达到 85% 以上。因此我认为，在这样的浅层网络上，模型可能不足以完全学会数据集的全部特征。

三、对于网络结构的理解

虽然卷积能够通过加深深度学习到更加高级的表征，这并不意味着越深的网络效果就越好。神经网络的复杂程度终究是与问题难度正相关的。通用近似定理已经告诉我们三层的神经网络在参数恰当的情况下能够以任意精度逼近任何连续函数，因而盲目地叠加神经网络的层数并不是一个好选择。随着网络的加深，训练的计算量会急剧上升，使得训练深层神经网络的开销非常巨大。如果不是问题难度对于深度的刚性需求，应该减少神经网络的层数。不仅如此，更深的神经网络可能会学习到数据中的噪声和细节，这并不是我们需要的，因

为这种噪声和细节不一定在新的数据集上仍然存在，从而导致模型泛化能力变弱。这也是深层神经网络在较为简单的问题上容易过拟合的原因。为了解决这种问题，人们引入了各种方式，如 `weight decay` 来限制数值过大的参数。